Fine-tuning di Winnow-12B e JPT-9B come fact checker: panoramica

Questa cartella descrive, passo per passo, come trasformare due *decision model* aperti (**Winnow-12B** e **JPT-9B**) in un **fact checker claim–documento** generale: dato un testo e un'affermazione, entrambi in prosa, il modello restituisce la probabilità che il testo supporti *tutta* l'affermazione. Il documento è sempre fornito: non c'è retrieval.

Ogni step ha un file dedicato con obiettivo, procedura, esempi concreti, comandi e criteri per passare allo step successivo. Questa panoramica dice cosa si fa, in che ordine e perché.

1. Il problema in una pagina

Input del modello: uno stato JSON con due campi e una domanda tipizzata sì/no.

```
{"document": "The meeting was moved to Tuesday at 10:00.", "claim": "The meeting is on Tuesday."}
```

> Domanda: "Does `document` support `claim`? Use only the document as evidence."

Output: le probabilità delle due opzioni (A/B) lette dai logit di **un solo token**, in **una sola passata** del modello. Esempio: P(supportato) = 0,94.

Etichetta: 1 = il documento supporta *ogni parte* del claim; 0 = almeno una parte è contraddetta **oppure** non stabilita (manca l'evidenza).

Perché partire da un decision model e non da un LLM generico: Winnow e JPT sono già addestrati a dare probabilità calibrate su domande tipizzate qualsiasi. Il fine-tuning deve solo specializzarli sul fact checking, senza rompere questa interfaccia.

2. Da dove partiamo

Risultati già misurati in questa repo su LLM-AggreFact (test, 29.320 righe, 11 fonti; results/runs/dev_threshold_summary.md):

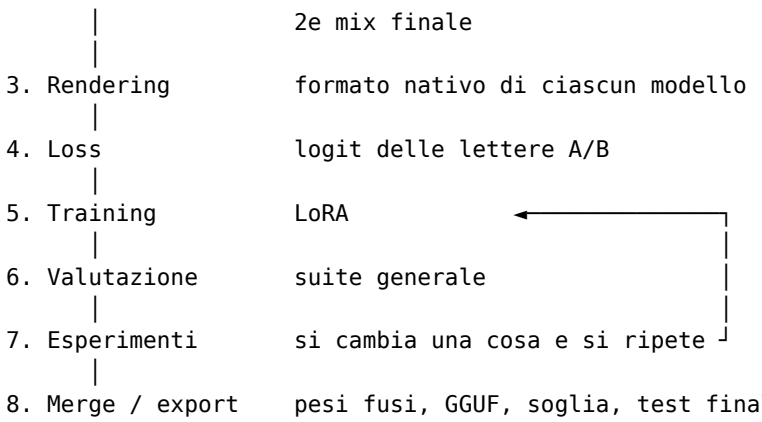
Modello	BAcc macro (soglia dal dev di AggreFact)	AUC macro	Licenza
jev-1.13.0 (riferimento, API chiusa)	79,09	86,89	—
Winnow-12B-BF16	77,42	85,02	Apache 2.0
JPT-9B	76,71	84,57	CC BY-NC 4.0
Bespoke-MiniCheck-7B (miglior modello aperto in leaderboard)	77,4		

Il divario con Jev è soprattutto di **ordinamento** (AUC più bassa), non di soglia: per colmarlo bisogna cambiare i pesi del modello. Gli errori si concentrano sui **riassunti** (notizie, trascrizioni di meeting) con errori sottili su entità e numeri.

Questi numeri sono "dev-tuned": la soglia è stata scelta sul dev di LLM-AggreFact. Con il protocollo di questa cartella (Step 0) LLM-AggreFact non entra più in nessuna decisione e viene valutato zero-shot; i numeri sopra restano come confronto secondario.

3. Il percorso in 10 step

```
0. Split e protocollo train / dev / test nostri, congelati; AggreFact solo test
      |
1. Baseline          numeri di partenza, senza training
      |
2. Dataset           2a unione delle fonti esistenti
      |              2b generazione con LLM
      |              2c replay contro l'oblio
      |              2d varianti di domanda
```



Step	File	Cosa produce	Stato nella repo
0. Split e protocollo	00b_split_e_protocollo.md	split train / dev_select / dev_confirm / test_id / test_ood congelati, senza AggreFact; regole di valutazione	coppie già pronte (`build_pairs.py`); split a tre vie da scrivere
1. Baseline	01_baseline.md	numeri di partenza di ciascun modello, con soglia e temperatura scelte sul nostro dev	in parte (cache di AggreFact già pronte)
2a. Unione delle fonti esistenti	02a_dataset_fonti_esistenti.md	coppie (documento, claim, etichetta) da 10 dataset, decontaminate (usato già dallo Step 0)	**fatto**: `data_prep/`
2b. Generazione con LLM	02b_dataset_generazione_llm.md	negativi difficili, FactKG in prosa, dati CG2C, verifica delle etichette	prototipo: `llm_gen/`
2c. Replay e oblio	02c_replay_e_forgetting.md	domande generiche con target del modello di partenza	da fare
2d. Varianti di domanda	02d_varianti_domanda.md	~40 parafrasi per domanda + nomi di campo alternativi	prototipo: `llm_gen/`
2e. Mix finale	02e_mix_finale.md	`train.jsonl` pronto (solo dallo split di train), con proporzioni e controlli	in parte (`make_mix.py`)
3. Rendering	03_rendering_formato.md	esempi tokenizzati nel prompt esatto di ciascun modello	da scrivere
4. Loss	04_loss.md	la funzione obiettivo: CE/Brier sulle lettere, KL di replay, termini contrastivi	da scrivere
5. Training	05_training.md	adapter LoRA addestrati, log, checkpoint	da scrivere
6. Valutazione	06_valutazione.md	test_id, test_ood, AggreFact zero-shot, test comportamentali, oblio	in parte (benchmark AggreFact)
7. Esperimenti	07_esperimenti.md	piano delle prove, ordine, criteri di decisione	piano
8. Merge, export, soglia	08_merge_export.md	pesi fusi, GGUF di Winnow, soglia congelata, pacchetto finale	da scrivere

Step	File	Cosa produce	Stato nella repo
Rischi e checklist	09_rischi_checklist.md	cosa può andare storto e una checklist per ogni step	—

4. Le decisioni chiave (e perché)

1. **LoRA, non full fine-tuning.** Pochi parametri, pesi originali intatti, entra su una GPU (L40S per Winnow, anche il portatile per JPT con QLoRA). Si può sempre tornare al modello di partenza.
2. **Formato nativo di ciascun modello.** I dati di training si renderizzano con le stesse funzioni dei benchmark della repo, token per token. Un prompt diverso farebbe imparare un formato che in uso non si vedrà.
3. **Loss solo sulle due lettere di risposta.** In inferenza si leggono due logit; si addestrano quei due logit. Un SFT classico sul vocabolario intero rende il modello sovra-confidente (esempio numerico in 04_loss.md).
4. **Dati testo contro testo.** Solo documenti in prosa: FactKG entra solo dopo essere stato trasformato in prosa, CG2C (FactCG) usa il grafo solo internamente.
5. **Replay contro l'oblio.** 10–15% di domande generiche con target presi dal modello di partenza (self-distillation): il modello resta un decision model generale.
6. **Valutazione su una suite, non su un solo benchmark.** Tre blocchi di test (in distribuzione, fuori dominio, LLM-AggreFact), più test comportamentali e benchmark di decisione.
7. **JPT per iterare, Winnow come candidato finale.** JPT è più piccolo (prove veloci sul portatile); Winnow ha AUC più alta e licenza Apache 2.0.
8. **Controllo con i modelli base.** Lo stesso training su Gemma-4-12B e Qwen3.5-9B base dice se partire da un decision model serve davvero.
9. **Tutte le decisioni sul nostro dev, LLM-AggreFact solo come test.** Soglia, temperatura, checkpoint ed esperimenti si scelgono su `dev_select` e si confermano su `dev_confirm`. LLM-AggreFact (dev compreso) non entra in nessuna scelta: il suo risultato è zero-shot e confrontabile con la leaderboard.

5. Obiettivi misurabili

Obiettivo	Soglia
LLM-AggreFact, BAcc macro **zero-shot** (soglia dal nostro dev)	$> 77{,}4$ (Bespoke-MiniCheck-7B, anch'esso zero-shot) con margine; puntare a ~79 (Jev, che però era dev-tuned)
Media dei tre blocchi di test (test_id, test_ood, AggreFact)	migliore della baseline dello stesso modello
test_ood (SciFact, Climate-FEVER, HealthVer, HaluBench, ...)	nessun peggioramento rispetto alla baseline
Oblio (JevBench, Kev-v9, typed-decisions)	calo ≤ 2 punti per benchmark
Calibrazione (ECE)	non peggiore della baseline
Test comportamentali	"solo claim" $\ll 0{,}5$; rimozione dell'evidenza fa scendere la probabilità in $> 80\%$ dei casi

6. Tempi e risorse (stime)

Fase	Macchina	Tempo indicativo
Split e protocollo (step 0)	qualsiasi	minuti di calcolo, una volta scritto lo script
Baseline (step 1)	L40S	poche ore di inferenza (dev e test nostri; AggreFact dalle cache esistenti)
Dati esistenti (2a)	qualsiasi	minuti (già fatto)
Generazione con LLM (2b–2d)	L40S con un modello locale, oppure API	da poche ore a 1–2 giorni, secondo il volume
Pilota su JPT (step 5)	portatile 24 GB (QLoRA) o L40S	5–10 h per corsa
Winnow (step 5)	L40S 45 GB (2× per data parallel)	6–20 h per corsa

Fase	Macchina	Tempo indicativo
Valutazione completa (step 6)	L40S	2–6 h per modello

7. Mappa della repo

Percorso	Ruolo
`data_prep/build_pairs.py`, `data_prep/make_mix.py`	step 0, 2a e 2e (lo split a tre vie è da aggiungere)
`llm_gen/`	prototipo per 2b, 2c, 2d (API NVIDIA)
`jpt-9b/benchmark.py`, `winnow-12b-bf16/benchmark.py`	prompt di riferimento (step 3) e valutazione (step 6)
`dev_split.py`, `calibrate_transfer.py`, `metrics.py`	pool del dev di AggreFact (ora solo per la soglia secondaria), applicazione di soglie alle cache, metriche
`winnow-12b-bf16/model/`	pesi bf16 di Winnow, base del fine-tuning
`finetune/` (da creare)	render, train, merge, valutatori

8. Documenti precedenti

Questa cartella riorganizza ed estende ../finetuning_strategy.md e ../dataset_finetuning.md, che restano validi come riferimento. In caso di differenze, vale questa cartella (più recente).